innerWidth / innerHeight

获取页面可视区域的宽度：

window.innerWidth

window.innerHeight

clientWidth / clientHeight

跟文档相关的，滚动条不属于文档，它属于浏览器的，属于窗口的。

document.documentElement.clientWidth / clientHeight //不包含滚动条

获取文档可视区域的宽度与高度

console.log(document.documentElement.clientWidth);

console.log(document.documentElement.clientHeight);

//没有兼容性的问题，IE8要再宽4个px;

padding + contentWidth /contentHeight (样式宽与样式高+ padding)

包含元素的padding ， 不包含border;

offsetWidth / offsetHeight

整个盒子的占位大小 padding + border：

//padding + border + 样式宽 / 样式高 -> 元素的实际占位大小

scrollHeight

获取ele的padding + width / height ， 但是如果子元素超过父元素宽度时，会加上超出部分的宽高;

```
//宽高 + padding + 超出的子元素的宽度与高度
<div id='box'>
    1<br/>
    1<br/>
    1<br/>
    1<br/>
    1<br/>
    1<br/>
    1<br/>
</div>
alert( box.scrollHeight );
/*
    但是如果子元素超过父元素宽度时，不受overflow:hidden的影响;
    只管当前对象，不管子元素。
*/
```

offsetTop / offsetLeft

获取ele元素到定位父级的top和left方向的距离（ ）

scollTop / ScrollLeft

获取元素的滚动距离，滚动了多少距离。也就打印多少。

鼠标事件clientX / clientY

获取鼠标到文档可视区域的top和left方向的距离 ；

//没有兼容性的问题！

document.onclick = function(e){ //e为事件源 、 为鼠标事件对向; console.log(e)

console.log(e.clientX);

console.log(e.clientY);

}

//注意：是文档可视区而不是页面的顶部区域;

```
//事件对象event
        document.onclick = function( e ){
            //获取鼠标到文档可视化区域的top与left;
            console.log( e )
            console.log( e.clientX , e.clientY )
            //获取鼠标到文档不可视化区域的top与left;
            console.log( e.pageX , e.pageY )

        }
```

获取子元素到浏览器top left的距离：

详细看源码19 CSSOM视图模式

offsetParent

offsetParent属性返回一个对象的引用，这个对象是距离调用offsetParent的元素最近的（在包含层次中最靠近的），并且是已进行过CSS定位的容器元素。

获取内部样式

```
getComputedStyle(Obj)
```

```
Ojb.currentStyle
```

主流浏览器能运行，IE8及以上也可以

```
            height:200px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }
        .wrap1 {
            width:450px;
            height:400px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }
        .wrap2 {
            width:60px;
            height:60px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }

    </style>
</head>
<body>


    <div class="wrap" id='wrap' style=''></div>
    <div class="wrap1" id='wrap1' style=''></div>
    <div class="wrap2" id='wrap2' style=''></div>



    <script>

        var oWrap = document.getElementById('wrap');
        var oWrap1 = document.getElementById('wrap1');
        var oWrap2 = document.getElementById('wrap2');
        //获取样式;
        // 优化1:
        function getStyle( ele ){
            console.log( ele )
            var cssObj ;
            if( ele.currentStyle ){
                //不是undefined;  ie
                cssObj = oWrap.currentStyle;
            }else{
                //undefied;   主流浏览器【火狐浏览器, 谷歌, QQ,傲游, , , 】
                cssObj = getComputedStyle(ele);
            }
        }
        console.log( cssObj )

        //优化2:
        function getStyle( ele ){
            // console.log( ele )
            var cssObj ;
            //三目:
            // ele.currentStyle ?cssObj = oWrap.currentStyle:cssObj = getComputedStyle(ele);
            return cssObj = ele.currentStyle ? ele.currentStyle: getComputedStyle(ele)

        }

        //优化3:
        function getStyle( ele ){
            // console.log( ele )
            //undefined
            //三目:
            // ele.currentStyle ?cssObj = oWrap.currentStyle:cssObj = getComputedStyle(ele);
            return   ele.currentStyle ||  getComputedStyle(ele)

            // 如果是主流浏览器, ele.currentStyle就是undefined,undefined为假值, 走或取后面的getComputedStyle(ele);
            // 如果为IE浏览器, ele.currentStyle为真, 不会走 || 取后的内容;
        }



        console.log( getStyle( oWrap ).width ,getStyle( oWrap ).height )//200.5px 200px
        console.log( getStyle( oWrap1 ).width ,getStyle( oWrap1 ).height )//450px 400px
        console.log( getStyle( oWrap2 ).width ,getStyle( oWrap2 ).height )//60px 60px
```